

Assessing Software Product Quality in DevOps: An ISO 25010:2023 Perspective

Saima Rafi
Edinburgh Napier University
Edinburgh, United Kingdom
s.rafi@napier.ac.uk

Muhammad Azeem Akbar
LUT University Finland
Lappeenranta, Finland
azeem.akbar@lut.fi

Arif Ali Khan
University of Oulu
Oulu, Finland
arif.khan@oulu.fi

Abstract

DevOps has revolutionised traditional development workflows by integrating continuous deployment, automated testing, and real-time monitoring, yet its impact on structured software quality assessment remains underexplored. This study evaluates DevOps-based software development through the lens of ISO 25010:2023, analysing how key DevOps practices—including automation, deployment, monitoring, and security strategies—influence critical software quality attributes. By systematically assessing these interactions, this research identifies the strengths and limitations of DevOps in achieving high-quality software and highlights the challenges organisations must address to bridge the gap between agility and structured quality assurance. The findings provide actionable insights into optimising DevOps workflows while ensuring alignment with ISO 25010 standards, paving the way for more reliable, maintainable, and performance-driven software development.

CCS Concepts

• DevOps → Software Product Quality; • ISO/IEC 25010:2023 → analysis.

Keywords

DevOps workflows, Software product quality

ACM Reference Format:

Saima Rafi, Muhammad Azeem Akbar, and Arif Ali Khan. 2025. Assessing Software Product Quality in DevOps: An ISO 25010:2023 Perspective. In *Proceedings of the 2025 Evaluation and Assessment in Software Engineering Companion (EASE Companion '25)*, June 17–20, 2025, Istanbul, Türkiye. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3727967.3756847>

1 Introduction

The increasing demand for faster software development cycles, high system availability, and continuous feature deployment has led organisations to adopt DevOps as a key approach to modern software engineering. DevOps integrates software development (Dev) and IT operations (Ops) to improve collaboration, automation, and efficiency, ensuring seamless software delivery, testing, and deployment[11]. Unlike traditional software development methodologies, which often suffer from long release cycles[24], fragmented workflows[7], and operational bottlenecks[40], DevOps fosters

continuous integration, continuous deployment (CI/CD), and real-time system monitoring, enabling higher agility and operational stability[32, 35].

DevOps has proven to enhance efficiency and speed in software development, but its impact on software product quality remains an active area of research and consideration [31, 32]. Quality in software development is typically assessed using standardised models such as ISO/IEC 25010, which defines software product quality across nine key characteristics: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability [10]. DevOps, through its reliance on automation, proactive monitoring, and iterative deployments, has the potential to influence multiple software quality attributes. For example, automated testing and continuous deployment improve reliability and maintainability, while real-time monitoring and cloud-based infrastructure contribute to performance efficiency and security [2].

Despite these advantages, implementing DevOps in software development comes with challenges. Many organisations struggle with standardising DevOps practices, mitigating security risks, and balancing automation with governance [29]. Additionally, while DevOps enhances certain quality attributes, its impact on other key characteristics remains underexplored, particularly in areas such as functional correctness, resource utilisation, and interoperability[25, 41]. Understanding the relationship between DevOps practices and software product quality is essential for optimising DevOps adoption while ensuring high-quality software delivery.

This study evaluates DevOps-based software development through the lens of ISO 25010:2023 by assessing how software development quality attributes are influenced by DevOps practices. By systematically analysing DevOps practices, this paper explores key factors that impact software quality, including automation, deployment, monitoring, and security strategies. Through this evaluation, the study provides a structured assessment of DevOps' role in shaping software product quality, highlighting its strengths, limitations, and the key challenges organisations must address to achieve high-quality software development through DevOps.

2 ISO/IEC 25010 and Its Role in Software Quality Evaluation

ISO/IEC 25010 is an internationally recognised standard that provides a comprehensive framework for evaluating software product quality. It defines a structured model that categorises software quality into nine key characteristics: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, safety, and portability. Each characteristic is further divided into sub-characteristics, allowing for a more granular assessment of software quality attributes. The standard is an essential reference



This work is licensed under a Creative Commons Attribution 4.0 International License. *EASE Companion '25, Istanbul, Türkiye*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1832-8/25/06
<https://doi.org/10.1145/3727967.3756847>

for software developers, quality assurance teams, and researchers, offering a systematic approach to assessing, comparing, and improving software products based on well-defined quality criteria [10].

The need for ISO/IEC 25010 arises from the increasing complexity of modern software systems, where ensuring high-quality, reliable, and secure applications is a critical challenge. As software development methodologies evolve—especially with the adoption of DevOps, Agile, and cloud-native architectures—quality assessment must align with dynamic and iterative software delivery models [41]. Traditional quality models often struggle to keep pace with DevOps-driven environments' rapid and iterative development cycles, where continuous integration, deployment, and automation reshape the software lifecycle. In this context, ISO/IEC 25010 serves as a valuable framework for evaluating software quality in real-world applications, providing a structured approach to measuring key quality attributes amid evolving development practices. By offering a standardised approach to measuring quality, ISO/IEC 25010 facilitates benchmarking, regulatory compliance, and continuous improvement in software engineering practices [7].

Researchers have increasingly utilised ISO/IEC 25010 to assess software quality in various domains, including Low-Code/No-Code development[28], DevOps [7], AI-driven applications [30], and cloud computing [37]. However, the extent to which these studies fully adhere to the framework varies. Some research efforts directly apply the standard's characteristics and sub-characteristics to evaluate software products, while others use modified or partial adaptations to fit specific research contexts. For example, studies on DevOps-based quality assessment primarily emphasise reliability, maintainability, and performance efficiency, as these attributes are directly influenced by automation, continuous integration, and deployment processes [7, 31, 39]. In contrast, research in cybersecurity and regulatory compliance often references ISO/IEC 25010 for security and confidentiality measures, highlighting the need for robust access control and data protection [13]. While ISO/IEC 25010 remains a widely accepted framework for software quality evaluation, some studies suggest that it must evolve to accommodate AI-driven development, cloud-native architectures, and automation-driven workflows in modern DevOps environments[7, 29, 41].

However, despite the growing reliance on DevOps for accelerating software delivery and improving operational efficiency, its impact on software product quality has not been systematically assessed using ISO/IEC 25010:2023. Existing research explores specific quality attributes, such as reliability and maintainability, but often lacks a comprehensive evaluation of DevOps' influence across all ISO 25010 characteristics, particularly security, functional correctness, and long-term maintainability. This limitation is further emphasised by the reliance on ISO 9126 and ISO/IEC 25010:2011 for software quality assessment [31, 39], as these standards were developed before the widespread adoption of automation-driven, iterative, and real-time monitoring workflows in DevOps. As a result, their applicability to modern DevOps environments remains an area of ongoing discussion, particularly regarding their ability to comprehensively assess quality attributes influenced by continuous deployment and integration practices. Furthermore, many studies focus on case-specific implementations without providing a structured, standardised assessment framework applicable across diverse DevOps environments. Given the increasing adoption of DevOps,

a comprehensive evaluation of its influence on software quality through ISO 25010:2023 is essential. This study aims to bridge this gap by examining how DevOps practices align with ISO 25010:2023, offering insights into its strengths, limitations, and areas requiring further optimisation to ensure high-quality software development in DevOps environments.

3 DevOps: ISO 25010:2023 Quality Perspective

Ensuring software quality in DevOps-driven development requires a structured framework to evaluate how automation, rapid deployments, and continuous integration affect different quality attributes. The ISO 25010:2023 model provides a standardised method for assessing software product quality, ensuring that DevOps processes align with key quality aspects such as functional suitability, performance efficiency, security, maintainability, and reliability [10].

While DevOps enhances deployment speed and automation, these benefits also introduce risks such as functional correctness issues, security vulnerabilities, and maintainability concerns if quality controls are not properly integrated into CI/CD pipelines [7, 25]. Frequent releases, shortened testing cycles [17], and real-time updates can increase the risk of undetected defects in production, making shift-left testing, regression testing, and contract testing essential to mitigating these risks [11, 26].

The ISO 25010:2023 framework allows organisations to systematically assess software quality across nine key characteristics, ensuring that rapid software delivery does not compromise critical quality attributes. However, research indicates that many DevOps implementations prioritise speed and automation over structured quality assessments in areas such as security, maintainability, usability, and flexibility [17, 22]. To bridge these gaps, organisations should integrate ISO 25010:2023-based quality gates into CI/CD pipelines, ensuring that DevOps-driven development meets standardised quality benchmarks while maintaining agility and efficiency.

(1) **FUNCTIONAL SUITABILITY:** Functional Suitability ensures that the software system correctly implements required features and meets user expectations.

- Functional correctness is achieved by integrating automated testing into CI/CD pipelines, which continuously validate system behavior against specifications [7].
- Functional completeness ensures that all expected features are available, preventing missing functionalities that could disrupt workflows. Functional appropriateness as a part of functional completeness ensures that all features are available and measures, whether implemented, features effectively serve user needs, which is enhanced by continuous feedback and iterative development processes [4].

However, challenges arise when automated pipelines fail to cover all functional scenarios, leading to undetected defects that may compromise business processes. This limitation underscores the need for comprehensive test coverage, including edge cases and real-world scenarios, to ensure reliability in production environments.

(2) **PERFORMANCE EFFICIENCY:** Performance efficiency focuses on a system's ability to manage workload demands

while maintaining optimal speed and resource usage. DevOps practices ensure high performance through continuous monitoring, automation, and dynamic scaling.

- Time behavior is improved through continuous performance testing, which monitors application response times and optimises bottlenecks before they impact production. Automated testing frameworks and real-time observability tools help monitor latency, throughput, and error rates, ensuring that performance remains stable under varying loads [26].
- Resource utilisation is enhanced with cloud-native solutions and container orchestration tools like Kubernetes, which dynamically allocate computing resources based on workload demand [41]. These technologies allow workloads to be efficiently distributed across servers, reducing resource wastage and improving cost efficiency.
- Capacity scalability ensures that systems can scale seamlessly to accommodate high traffic without affecting performance [3].

These advancements are valuable, but performance trade-offs are still an open challenge. Frequent deployments, misconfigured auto-scaling policies, and inadequate testing for extreme loads can lead to unexpected slowdowns in production. Organisations must implement thorough load testing, efficient resource allocation, and proactive monitoring to ensure that DevOps pipelines deliver sustained performance improvements [7].

- (3) **COMPATIBILITY:** Compatibility in DevOps measures how well applications integrate across diverse environments while maintaining stability and interoperability. Given the distributed nature of modern software systems, ensuring seamless interactions between microservices, third-party services, and legacy infrastructure is crucial for sustained performance and reliability.
 - Coexistence ensures that software does not interfere with other applications running within the same ecosystem, which is maintained through containerisation and service isolation [29]. For instance, Docker and Kubernetes play a key role in maintaining coexistence by isolating applications and their dependencies, ensuring they run consistently across different environments [25].
 - Interoperability is ensured by adopting standardised API development, enabling seamless communication between different microservices and external systems. RESTful APIs, GraphQL, and gRPC allow different microservices and external systems to communicate efficiently while maintaining platform independence [33]. API gateways such as Kong, Apigee, and AWS API Gateway enhance interoperability by managing authentication, request routing, and throttling, ensuring reliable service communication [42].

While DevOps fosters high compatibility, challenges arise when frequent deployments lead to breaking changes in API structures or microservices dependencies [1]. Backward compatibility strategies such as versioning, contract testing, and feature flagging help mitigate these risks, ensuring continuous service availability [31].

- (4) **USABILITY:** Usability in DevOps is crucial for both developers and end-users who interact with applications and deployment environments. Learnability, operability, and error prevention are key factors that enhance the user experience in DevOps-driven workflows.

- Learnability is improved through intuitive DevOps tools, such as self-service deployment interfaces that simplify operational workflows [21]. Continuous integration (CI) tools such as Jenkins and GitLab, along with infrastructure-as-code solutions like Terraform, simplify onboarding and configuration management, ensuring that new users can quickly learn and adapt [20].
- Operability is enhanced by providing automated monitoring dashboards that enable teams to track software performance and troubleshoot issues efficiently. Tools such as Prometheus, Grafana, and ELK Stack provide visibility into system health, allowing DevOps teams to identify and resolve performance issues efficiently [12].
- User error prevention is achieved through automated testing, deployment safeguards, and policy-driven access controls. Test-driven development (TDD) and behavior-driven development (BDD) methodologies enforce strict validation processes, ensuring that software changes meet functional and security requirements before deployment [38, 45].

DevOps enhances usability; however, balancing automation complexity with human oversight remains a challenge, as excessive reliance on automated pipelines can obscure debugging insights, making failures harder to diagnose—necessitating the integration of interactive logging, AI-driven analytics, and adaptive testing strategies to maintain efficiency and usability [25].

- (5) **RELIABILITY:** Reliability ensures that software remains operational and available under various conditions, a fundamental goal in DevOps-driven development. Maturity, fault tolerance, and recoverability are key components that contribute to system reliability.
 - Maturity is enhanced through well-established DevOps practices such as automated rollback mechanisms and canary deployments, which reduce downtime during updates. These strategies allow new releases to be tested in real environments while ensuring quick rollback options in case of failures. [5].
 - Fault tolerance is strengthened by implementing redundant system architectures that maintain uptime even when components fail [41]. Additionally, self-healing infrastructure using automated monitoring and orchestration tools like Kubernetes helps maintain uptime even during system failures [39].
 - Recoverability is improved through automated disaster recovery solutions and infrastructure-as-code practices that enable quick restoration of system states. Tools like Terraform and Ansible facilitate rapid restoration of system states, reducing downtime in case of outages [19, 20].
- Despite advantages, there are challenges such as frequent deployments, dependency updates, and cloud service fluctuations which can introduce unexpected failures [40]. To

address this, organisations must implement chaos engineering, predictive analytics, and continuous resilience testing to strengthen system reliability [9].

- (6) **SECURITY:** Security in DevOps is a continuous process integrated into the development lifecycle, often referred to as DevSecOps. Security in DevOps focuses on confidentiality, integrity, and non-repudiation to ensure that applications remain protected from unauthorised access, data breaches, and tampering.

- Confidentiality is ensured by enforcing access control policies, encrypting sensitive data, and integrating security scans into CI/CD pipelines.
- Integrity is upheld through version-controlled infrastructure and cryptographic validation of deployed code, preventing unauthorised modifications and ensuring the authenticity of software components [34, 43].
- Non-repudiation is achieved by implementing immutable logging, detailed audit trails, and compliance monitoring, ensuring accountability for all system changes and reducing the risk of tampering [15].

DevSecOps promotes continuous security testing, where automated vulnerability assessments and compliance checks are integrated into the development pipeline [36]. By embedding security at every stage, DevOps teams proactively mitigate risks, ensuring software remains resilient against threats while maintaining development agility. integrated into the development lifecycle [34].

- (7) **MAINTAINABILITY:** Maintainability in DevOps ensures that software can be modified, enhanced, and debugged efficiently. Modularity, reusability, and analysability are key components that contribute to software maintainability.

- Modularity is achieved through microservices architectures, enabling teams to update individual components without affecting the entire system. This approach improves scalability and minimises dependencies, reducing the risks associated with large-scale changes [18].
- Reusability is enhanced through infrastructure automation and the use of pre-configured deployment templates, allowing teams to standardise environments and streamline updates across multiple projects. This reduces redundancy and ensures consistency in software operations [27].
- Analysability is supported by monitoring tools and logging solutions that provide real-time insights into software performance and errors [14]. Automated observability frameworks, including log aggregation and alerting mechanisms, enable quick identification and resolution of issues, thereby improving system stability and reducing downtime caused by undetected failures [23].

Despite these advancements, maintaining software across frequent releases poses challenges. Ensuring consistency in microservices-based architectures requires robust documentation, dependency management, and automated regression testing [14]. Implementing self-healing infrastructure, AI-driven error analysis, and predictive maintenance can further enhance software maintainability in DevOps environments[27].

- (8) **SAFETY:** Safety in DevOps ensures that systems can recover from failures, minimise disruptions, and maintain operational integrity under various conditions. Fail-safe mechanisms, risk assessment, and resilience strategies play a critical role in ensuring safety in DevOps environments.

- DevOps integrates automated rollback strategies, backup policies, and disaster recovery solutions to prevent catastrophic failures. By using continuous monitoring, real-time alerts, and self-healing infrastructure, organisations can detect and mitigate failures before they escalate [27, 44].
- Proactive risk assessment in DevOps leverages predictive analytics, incident management frameworks, and security compliance checks to identify potential vulnerabilities. Organisations implement chaos engineering [9] and failure injection testing to evaluate system resilience and ensure readiness for unexpected events [40].
- To maintain stability under high-demand conditions, auto-scaling, redundancy planning, and distributed system architectures are employed. These approaches ensure business continuity, improve uptime, and reduce the likelihood of major outages [16].

Although using these measures, fast deployment cycles and frequent updates increase the risk of undetected failures. Organisations must integrate continuous resilience testing, automated fault detection, and AI-driven anomaly detection to maintain system reliability in DevOps environments[31].

- (9) **FLEXIBILITY and PORTABILITY:** Flexibility and portability in DevOps enables organisations to rapidly adapt to changing business requirements, technology landscapes, and operational demands. adaptability, scalability, interoperability and automation are key components that contribute in this area.

- Adaptability is achieved through the use of service-oriented architectures (SOA) and microservices, which allow teams to implement incremental changes without disrupting existing functionality[41]. This approach minimises downtime and enhances software agility, ensuring that updates, patches, and new features can be deployed efficiently .
- Scalability is another critical aspect of flexibility, enabling software systems to handle fluctuating workloads and increasing demand. DevOps leverages cloud-based infrastructure, container orchestration (such as Kubernetes), and auto-scaling mechanisms to dynamically adjust resource allocation. This ensures that applications remain responsive and cost-effective under varying levels of usage, supporting agile business expansion [26].
- Cloud-agnostic architectures and standardised APIs prevent vendor lock-in, ensuring that applications can run on multiple cloud platforms without significant changes[27, 37].
- Configuration as code and infrastructure as code (IaC) provides the flexibility to replicate and modify environments quickly [42], allowing for seamless transitions between the development, testing, and production phases. This level of automation reduces manual intervention, accelerates

Table 1: A summary of key points on DevOps from ISO 25010:2023 quality perspective

ISO 25010:2023 Quality Attribute	Key DevOps Considerations	Challenges
Functional Suitability	Automated testing in CI/CD ensures correctness; microservices enhance modular functionality.	Risk of incomplete test coverage leading to undetected defects in production.
Performance Efficiency	Continuous performance testing, auto-scaling, and container orchestration optimise response times.	Latency issues due to inefficient scaling or unexpected resource bottlenecks.
Compatibility	Containerisation, API standardisation, and service isolation maintain system interoperability.	Legacy system integration challenges and backward compatibility concerns.
Interaction Capability	Self-service deployment, automated monitoring, and reduced manual intervention enhance usability.	Over-reliance on automation may obscure debugging insights and error traceability.
Reliability	Automated rollback, fault-tolerant architectures, and disaster recovery solutions ensure stability.	Risks of failures during frequent deployments and unexpected downtime in complex workflows.
Security	Integrated security scans, immutable logging, and compliance automation protect system integrity.	Security vulnerabilities due to misconfigurations, dependency risks, and compliance gaps.
Maintainability	Microservices, infrastructure-as-code (IaC), and logging solutions improve debugging and updates.	Difficulty in managing dependencies and ensuring system stability across multiple services.
Flexibility/Portability	Service-oriented architectures (SOA), dynamic scaling, and automated provisioning enhance system flexibility. Additionally, cross-platform deployments and cloud-agnostic architectures support adaptability.	Complexity in cross-platform migration and potential failure risks in high-speed DevOps cycles.
Safety	Fail-safe mechanisms, automated backups, and rollback strategies prevent catastrophic failures. Risk assessment tools analyse potential threats and ensure system resilience.	Fast deployment cycles increase the risk of undetected failures, necessitating continuous resilience testing.

deployments, and enhances the ability to recover from failures or roll back changes when necessary [6].

Despite these advantages, managing flexibility in complex DevOps environments presents challenges, such as dependency management, configuration drift, and unpredictable scaling behaviors [8, 25]. Organisations can mitigate these risks by implementing continuous performance testing, automated rollback strategies, and predictive scaling models to ensure seamless adaptation to new requirements.

Therefore, by implementing DevOps based on ISO 25010:2023, organisations can achieve a scalable, secure, and high-performing software ecosystem. Each characteristic outlined above ensures that DevOps practices align with modern software engineering quality standards. The summary of the overarching key points

relevant to DevOps (discussed earlier in this section) from the perspective of ISO 25010:2023 is presented in Table 1.

4 COMPREHENSIVE ANALYSIS OF DEVOPS: TECHNOLOGICAL EVALUATION, CHALLENGES, AND FUTURE TRENDS

As DevOps continues to reshape modern software development, it becomes essential to assess its impact on key technological attributes, market adaptability, and long-term sustainability. By integrating automation, continuous integration/continuous deployment (CI/CD), and infrastructure-as-code (IaC), DevOps enhances software efficiency, scalability, and security. However, challenges related to compliance, interoperability, and governance remain, necessitating a structured evaluation of its strengths and limitations.

While compliance plays a significant role in regulated industries, DevOps extends beyond governance concerns, encompassing automation, observability, scalability, and AI-driven optimisations. This section examines DevOps in relation to ISO 25010:2023 product quality attributes, addressing its technological advancements, adoption challenges, and future trends.

- (1) **Technological Evaluation of DevOps:** DevOps has transformed software development by integrating automation, collaboration, and continuous delivery into software life-cycles. It enhances functional suitability through CI/CD pipelines, automated testing, and infrastructure-as-code (IaC), enabling rapid iterations and stable releases. Performance efficiency is achieved through containerisation, cloud-native architectures, and auto-scaling mechanisms that optimise workload distribution. Compatibility within multi-cloud and hybrid environments is strengthened by API-driven integrations and microservices, ensuring seamless interoperability across diverse platforms. However, DevOps still faces limitations in handling complex regulatory frameworks and security mandates. While automation reduces human error, it also increases risks associated with misconfigurations and supply chain vulnerabilities. Organisations must carefully balance efficiency, compliance, and reliability to maximise the effectiveness of DevOps methodologies.
- (2) **Challenges in DevOps Adoption and Market Adaptability:** DevOps still encounters technological and market-driven challenges for example, security vulnerabilities in CI/CD pipelines pose risks, as rapid deployments may introduce misconfigurations or unverified dependencies. Scalability in complex enterprise environments remains difficult, particularly in highly regulated industries requiring strict governance, traceability, and risk management. Another challenge is interoperability and vendor lock-in, where organisations adopting multi-cloud architectures struggle to enforce consistent security policies and maintain workload portability across cloud providers. Additionally, the growing emphasis on AI integration in DevOps raises concerns about data privacy, model security, and automation reliability. The cultural shift toward DevOps-first approaches also requires significant investments in training, collaboration, and organisational restructuring to ensure smooth adoption.
- (3) **Practical Considerations on the Use of DevOps in Regulated Industries:** To optimise DevOps adoption, organisations must focus on automation, governance, and adaptability. Embedding compliance into DevOps pipelines through automated security scans, policy-as-code, and real-time risk monitoring ensures adherence to industry standards while maintaining agility. Governance frameworks such as ISO 27001 and NIST should be seamlessly integrated into DevOps workflows to maintain regulatory alignment without creating bottlenecks in software delivery. Additionally, standardising DevOps governance models across multi-cloud environments will improve security posture and operational consistency. AI-driven observability tools can help detect anomalies and

automate root-cause analysis, reducing downtime and enhancing system resilience. Investing in team training and cross-functional collaboration will also accelerate DevOps maturity, ensuring that security, reliability, and compliance are embedded into every stage of development while allowing teams to retain the flexibility needed for continuous innovation.

- (4) **Future Trends:** The future of DevOps is shaped by automation, AI-driven optimisations, and enhanced governance mechanisms. AI and machine learning will play a significant role in automated incident resolution, anomaly detection, and predictive analytics, reducing the manual workload in DevOps operations. Self-healing systems will become more prevalent, where infrastructure can auto-remediate issues in real time without human intervention.

Additionally, blockchain-based audit trails will provide immutable compliance records, ensuring transparency and integrity in regulatory reporting. Cloud-native DevOps architectures will expand further, promoting serverless computing, edge deployments, and dynamic workload management. The integration of DevOps with FinOps will also optimise cost management in cloud environments, ensuring that resource allocations align with business needs.

As organisations scale their DevOps practices, standardisation efforts in multi-cloud compliance and governance will continue to evolve. Regulatory-as-code and compliance-as-code models will automate security enforcement, ensuring that compliance checks occur in real-time rather than as an afterthought. While compliance remains a critical consideration in DevOps governance, the emphasis will shift toward ensuring resilience, automation, and cross-platform adaptability to meet the evolving demands of modern enterprises.

5 Future Research Directions

Future research should explore the role of ISO standards in guiding the evolution of DevOps methodologies. With the increasing reliance on DevOps in highly regulated industries, compliance with frameworks such as ISO/IEC 27001, ISO/IEC 25010, and ISO/IEC 42010 will become critical in ensuring software quality, security, and architectural integrity. Future studies should assess how ISO standards impact DevOps governance models, influencing auditability, risk assessment, and software standardisation.

Additionally, research should investigate how organisations adapt DevOps practices to meet ISO-driven software quality requirements while maintaining agility and innovation. Comparative studies can analyse the effectiveness of ISO-certified vs. non-certified DevOps environments, providing insights into best practices for balancing software reliability, security, and efficiency. Exploring the role of ISO will also help in understanding how its characteristics can streamline software quality validation and risk mitigation in DevOps workflows. By integrating ISO frameworks into DevOps research, future studies can offer structured approaches to achieving scalable, secure, and standardised DevOps implementations in shaping DevOps scalability, security, and overall product quality.

Acknowledgments

This paper has used an AI-generated text at drafting stage of the writing process to refine text structure and align paragraphs. The authors reviewed, edited, and takes full responsibility for the content.

References

- [1] Fernando Almeida, Jorge Simões, and Sérgio Lopes. 2022. Exploring the benefits of combining devops and agile. *Future Internet* 14, 2 (2022), 63.
- [2] Ahmad Alnafessah, Alim Ul Gias, Runan Wang, Lulai Zhu, Giuliano Casale, and Antonio Filieri. 2021. Quality-aware devops research: Where do we stand? *IEEE access* 9 (2021), 44476–44489.
- [3] Ricardo Amaro, Rúben Pereira, and Miguel Mira da Silva. 2024. DevOps metrics and KPIs: a multivocal literature review. *Comput. Surveys* 56, 9 (2024), 1–41.
- [4] Sher Badshah, Arif Ali Khan, and Bilal Khan. 2020. Towards process improvement in DevOps: a systematic literature review. In *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering*. 427–433.
- [5] Antonia Bertolino, Guglielmo De Angelis, Antonio Guerriero, Breno Miranda, Roberto Pietrantuono, and Stefano Russo. 2023. DevOpRET: Continuous reliability testing in DevOps. *Journal of Software: Evolution and Process* 35, 3 (2023), e2298.
- [6] Robin Bolscher and Maya Daneva. 2019. Designing software architecture to support continuous delivery and DevOps: a systematic literature review. In *14th International Conference on Software Technologies, ICSoft 2019*. SCITEPRESS, 27–39.
- [7] Daniel Céspedes, Paula Angeleri, Karin Melendez, and Abraham Dávila. 2020. Software product quality in DevOps contexts: a systematic literature review. In *Trends and Applications in Software Engineering: Proceedings of the 8th International Conference on Software Process Improvement (CIMPS 2019)*. Springer, 51–64.
- [8] V Hemanth Das Chowdary, A Shanmukh, T Phaneendra Nikhil, B Sandeep Kumar, and Feroz Khan. 2024. DevOps 2.0: Embracing AI/ML, Cloud-Native Development, and a Culture of Continuous Transformation. In *2024 4th International Conference on Pervasive Computing and Social Networking (ICPCSN)*. IEEE, 673–679.
- [9] Sheryl I Drake. 2022. *An Exploratory Study: Chaos Engineering Integration Within a Devops Environment*. Ph.D. Dissertation. Marymount University.
- [10] International Organization for Standardization. 2023. *ISO/IEC 25010: 2023: Systems and Software Engineering —Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model*.
- [11] Gene Gotimer and Thomas Stiehm. 2016. Devops advantages for testing: Increasing quality through continuous delivery. *CrossTalk Magazine* (2016), 13–18.
- [12] Subrat Gupta. 2024. *The Art of DevOps Engineering*. Subrat Gupta.
- [13] Jonas Heilmann. 2020. Application Security Review Criteria for DevSecOps Processes.
- [14] Adha Hrusto. 2024. *Enhancing DevOps with Autonomous Monitors: A Proactive Approach to Failure Detection*. Lund University.
- [15] Tony Hsiang-Chih Hsu. 2018. *Hands-On Security in DevOps: Ensure continuous security, deployment, and delivery with DevSecOps*. Packt Publishing Ltd.
- [16] Ishita Jaju. 2023. Maximizing DevOps Scalability in Complex Software Systems: Maximizing DevOps Scalability in Complex Software Systems.
- [17] Daiju Kato, Ayumu Shimizu, and Hiroshi Ishikawa. 2022. Quality classification for testing work in DevOps. In *Proceedings of the 14th international conference on management of digital ecosystems*. 156–162.
- [18] Srinikhita Kothapalli, Aditya Manikyala, Hari Priya Kommineni, SSMLG Gudimetla Naga Venkata, Pavan Kumar Gade, Abhishekar Reddy Allam, Narayana Reddy Bommu Sridharlakshmi, Narasimha Rao Boinapalli, AR Onteddu, and RR Kundavaram. 2019. Code Refactoring Strategies for DevOps: Improving Software Maintainability and Scalability. *ABC Research Alert* 7, 3 (2019), 193–204.
- [19] Mikael Krief. 2022. *Learning DevOps: A comprehensive guide to accelerating DevOps culture adoption with Terraform, Azure DevOps, Kubernetes, and Jenkins*. Packt Publishing Ltd.
- [20] Mohamed Labourdy. 2021. *Pipeline as code: continuous delivery with Jenkins, Kubernetes, and terraform*. Simon and Schuster.
- [21] Josef Langerman and Wai Sze Leung. 2024. The effect of outsourcing and insourcing on Agile and DevOps. *Journal of Information Technology Teaching Cases* 14, 2 (2024), 192–199.
- [22] Alexandra LAPOINTE-BOISVERT, Sylvie TRUDEL, and Jean-Marc DESHARNAIS. 2024. Measurement In DevOps Standard: Proposed Improvements. (2024).
- [23] Ankur Mahida. 2024. Integrating Observability with DevOps Practices in Financial Services Technologies: A Study on Enhancing Software Development and Operational Resilience. *International Journal of Advanced Computer Science & Applications* 15, 7 (2024).
- [24] Simo Mäkinen. 2023. Increasing Release Frequency by Accelerating Software Development Cycles in Software Engineering. (2023).
- [25] Alok Mishra and Ziadoon Otaiwi. 2020. DevOps and software quality: A systematic mapping. *Computer Science Review* 38 (2020), 100308.
- [26] Ibrahim Ali Mohammed. 2018. A methodical mapping on the relationship between DevOps and software quality. *International Journal of Creative Research Thoughts (IJCRT)* www.ijrt.org. ISSN (2018), 2320–2882.
- [27] Gabriel Morais. 2023. Enabling Systematic Microservices Reuse Through DevOps.. In *CAiSE (Doctoral Consortium)*. 1–10.
- [28] Bilal Naqvi, Damian Kedziora, Lanqi Zhang, and Shola Oyediji. 2025. Quality of Low-Code/No-Code Development Platforms Through the Lens of ISO 25010: 2023. *Computer* 58, 3 (2025), 30–40.
- [29] Akalanka Nayanajith and Ruwan Wickramarachchi. 2024. Challenges affecting the successful adoption of DevOps practices: a systematic literature review. In *2024 4th International Conference on Advanced Research in Computing (ICARC)*. IEEE, 311–315.
- [30] Jesús Oviedo, Moisés Rodríguez, Andrea Trenta, Dino Cannas, Domenico Natale, and Mario Piattini. 2024. ISO/IEC quality standards for AI engineering. *Computer Science Review* 54 (2024), 100681.
- [31] Pulasthi Perera, Roshali Silva, and Indika Perera. 2017. Improve software quality through practicing DevOps. In *2017 seventeenth international conference on advances in ICT for emerging regions (ICTer)*. IEEE, 1–6.
- [32] Dan Port, Bill Taber, and Parisa Emkani. 2024. Investigating effectiveness and compliance to DevOps policies and practices for managing productivity and quality variability. *Journal of Systems and Software* 213 (2024), 112030.
- [33] RVS Praveen. 2024. *Data Engineering for Modern Applications*. Addition Publishing House.
- [34] Saima Rafi, Muhammad Azeem Akbar, and Sajjad Mahmood. 2024. Aligning Academic with Industrial Needs: Investigating DevOps. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 516–519.
- [35] Saima Rafi, Wu Yu, Muhammad Azeem Akbar, Sajjad Mahmood, Ahmed Alsanad, and Abdu Guma. 2021. Readiness model for DevOps implementation in software organizations. *Journal of Software: Evolution and Process* 33, 4 (2021), e2323.
- [36] Roshan N Rajapakse, Mansoor Zahedi, M Ali Babar, and Haifeng Shen. 2022. Challenges and solutions when adopting DevSecOps: A systematic review. *Information and software technology* 141 (2022), 106700.
- [37] Anderson Ravanello, Jean-Marc Desharnais, Luis Eduardo Bautista Villalpando, Alain April, and Abdelouahed Gherbi. 2014. Performance measurement for cloud computing applications using ISO 25010 standard characteristics. In *2014 Joint Conference of the International Workshop on Software Measurement and the International Conference on Software Process and Product Measurement*. IEEE, 41–49.
- [38] Jukka Riihimäki. 2019. Automatic testing and delivery in web development. (2019).
- [39] Lalit Mohan Saini et al. 2024. The Impact of Devops on Software Quality. In *2024 Third International Conference on Smart Technologies and Systems for Next Generation Computing (ICSSTN)*. IEEE, 1–6.
- [40] Mali Senapathi, Jim Buchan, and Hady Osman. 2018. DevOps capabilities, practices, and challenges: Insights from a case study. In *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering* 2018. 57–67.
- [41] Sumanth Tatineni. 2023. Applying DevOps Practices for Quality and Reliability Improvement in Cloud-Based Systems. *Technix international journal for engineering research (TIJER)* 10, 11 (2023), 374–380.
- [42] Berk Türetken. 2024. Enhancing Security with Cloud-based API Management: Best Practices and Implementation.
- [43] Akond Ashfaq Ur Rahman and Laurie Williams. 2016. Security practices in DevOps. In *Proceedings of the Symposium and Bootcamp on the Science of Security*. 109–111.
- [44] Sricharan Vadapalli. 2018. *DevOps: continuous delivery, integration, and deployment with DevOps: dive into the core DevOps strategies*. Packt Publishing Ltd.
- [45] Joakim Verona. 2018. *Practical DevOps: Implement DevOps in your organization by effectively building, deploying, testing, and monitoring code*. Packt Publishing Ltd.